3666. Minimum Operations to Equalize Binary String

Solved

Hard | Topics | Companies | Hint

You are given a binary string `s`, and an integer `k`.

In one operation, you must choose **exactly** `k` **different** indices and **flip** each `'0'` to `'1'` and each `'1'` to `'0'`.

Return the **minimum** number of operations required to make all characters in the string equal to `'1'`. If it is not possible, return -1.

Example 1:

> **Input:** `s = "110", k = 1`
>
> **Output:** 1
>
> **Explanation:**
>
> - There is one `'0'` in `s`.
> - Since `k = 1`, we can flip it directly in one operation.

Example 2:

> **Input:** `s = "0101", k = 3`
>
> **Output:** 2
>
> **Explanation:**
>
> One optimal set of operations choosing `k = 3` indices in each operation is:
>
> - **Operation 1**: Flip indices `[0, 1, 3]`. `s` changes from `"0101"` to `"1000"`.
> - **Operation 2**: Flip indices `[1, 2, 3]`. `s` changes from `"1000"` to `"1111"`.
>
> Thus, the minimum number of operations is 2.

Example 3:

> **Input:** `s = "101", k = 2`
>
> **Output:** −1
>
> **Explanation:**
>
> Since `k = 2` and `s` has only one `'0'`, it is impossible to flip exactly `k` indices to make all `'1'`. Hence, the answer is -1.

Constraints:

- `1 <= s.length <= 10^5`
- `s[i]` is either `'0'` or `'1'`.
- `1 <= k <= s.length`

Python:

```python
class Solution:
    def minOperations(self, s: str, k: int) -> int:
        zero = 0
        len_s = len(s)

        for i in range(len_s):
            zero += ~ord(s[i]) & 1

        if zero == 0:
            return 0

        if len_s == k:
            return ((1 if zero == len_s else 0) << 1) - 1

        base = len_s - k

        odd = max(math.ceil(zero / k), math.ceil((len_s - zero) / base))

        odd += ~odd & 1

        even = max(math.ceil(zero / k), math.ceil(zero / base))

        even += even & 1

        res = float('inf')

        if (k & 1) == (zero & 1):
            res = min(res, odd)

        if (~zero & 1) == 1:
            res = min(res, even)

        return -1 if res == float('inf') else res
```

JavaScript:

```javascript
function minOperations(s, k) {
    const n = s.length;
    if (n < k) return -1;

    const seen = Array(n + 1).fill(false);
    const last = Array.from({ length: n + 1 }, (_, i) => i);

    function getLast(x) {
```

```
    if (last[x] !== x) last[x] = getLast(last[x]);
    return last[x];
}

let cnt0 = 0;

for (let i = 0; i < n; i++) {
    if (s[i] === "0") cnt0++;
}

if (cnt0 === 0) return 0;
if (cnt0 % 2 === 1 && k % 2 === 0) return -1;

seen[cnt0] = true;

class Deque {
    constructor() {
        this.data = [];
        this.head = 0;
    }

    pushBack(item) {
        this.data.push(item);
    }

    popFront() {
        if (this.head >= this.data.length) return undefined;

        const val = this.data[this.head];
        this.head++;

        if (this.head > 50 && this.head * 2 > this.data.length) {
            this.data = this.data.slice(this.head);
            this.head = 0;
        }
        return val;
    }

    isEmpty() { return this.head >= this.data.length; }
}

const q = new Deque();
q.pushBack([cnt0, 0]);
```

```
    while (!q.isEmpty()) {
        const cur = q.popFront();
        const [cur0, ops] = cur;
        const lo = Math.max(0, cur0 + k - n);
        const hi = Math.min(k, cur0);
        const new0_hi = cur0 + k - 2 * lo;
        const new0_lo = cur0 + k - 2 * hi;
        let new0 = getLast(new0_lo);

        while (new0 <= new0_hi) {
            if (new0 === 0) return ops + 1;

            if (!seen[new0]) {
                seen[new0] = true;
                q.pushBack([new0, ops + 1]);
            }

            if (new0 + 2 < last.length) {
                last[new0] = getLast(new0 + 2);
                new0 = last[new0];
            }

            else new0 += 2;
        }
    }

    return -1;
}
```

Java:

```
class Solution {
    public int minOperations(String s, int k) {
        int zero = 0;
        int len = s.length();

        for (int i = 0; i < len; i++)
            zero += ~s.charAt(i) & 1;

        if (zero == 0)
            return 0;

        if (len == k)
            return ((zero == len ? 1 : 0) << 1) - 1;
```

```
        int base = len - k;

        int odd = Math.max(
            (zero + k - 1) / k,
            (len - zero + base - 1) / base
        );

        odd += ~odd & 1;

        int even = Math.max(
            (zero + k - 1) / k,
            (zero + base - 1) / base
        );

        even += even & 1;

        int res = Integer.MAX_VALUE;

        if ((k & 1) == (zero & 1))
            res = Math.min(res, odd);

        if ((~zero & 1) == 1)
            res = Math.min(res, even);

        return res == Integer.MAX_VALUE ? -1 : res;
    }
}
```